Support de Cours
Module : Conception par contrats
Chapitre 4: OCL pour la conception par contrat

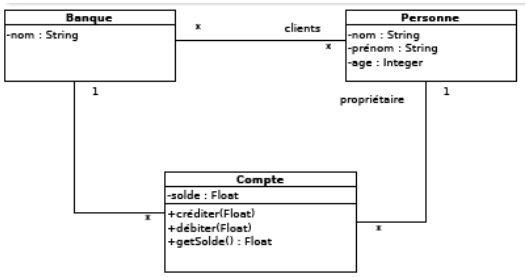
Réalisé par:
Dr. Sakka Rouis Taoufik

Introduction

Contraintes UML

Spécification d'une application bancaire

- *Un compte doit avoir un solde toujours positif*
- *Un client peut posséder plusieurs comptes*
- *Un client peut être client de plusieurs banques*
- *Un client d'une banque possède au moins un compte dans cette banque*
- *Une banque gère plusieurs comptes*
- *Une banque possède plusieurs clients*



Quelles spécifications ne sont exprimées ?

Introduction

Contraintes UML

➔ *Le diagramme de classe ne permet pas d'exprimer tout ce qui est déni dans la spécification*

➔ ***Solution***

Ajout de contraintes à des éléments de modélisation.

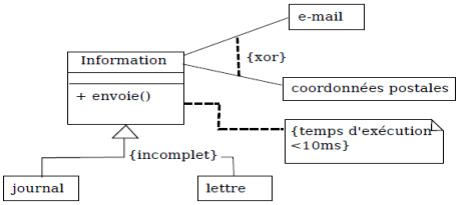
3

Introduction

Contraintes UML

Relation entre éléments de modélisation : propriété qui doit être vraie

- Notation : …{contrainte}
- 3 types de contraintes :
- Contraintes prédéfinies : {ordonné} {sous ensemble} {xor} {addOnly} {frozen}
- Contraintes exprimées en langue naturelle (commentaires)
- Contraintes exprimées avec OCL (Object Constraint Language)



4

Introduction

Contraintes UML sur les associations sans utiliser OCL

Contraintes sur les associations : Contrainte {ordonné}

Indique que les objets seront ordonnés à chaque opération de création, modification, suppression, …

Les comptes d'une personne sont ordonnés

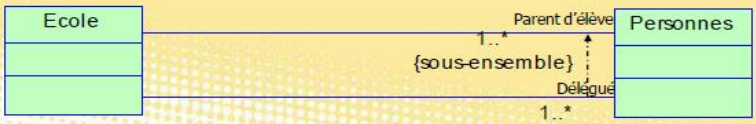
5

Introduction

Contraintes UML sur les associations sans utiliser OCL

Contraintes sur les associations : Contrainte {sous-ensemble}

- Indique qu'une collection est incluse dans une autre
- Nécessite la présence d'au moins deux relations



Les personnes qui jouent le rôle de délégué font partie des personnes qui jouent le rôle de parents d'élèves

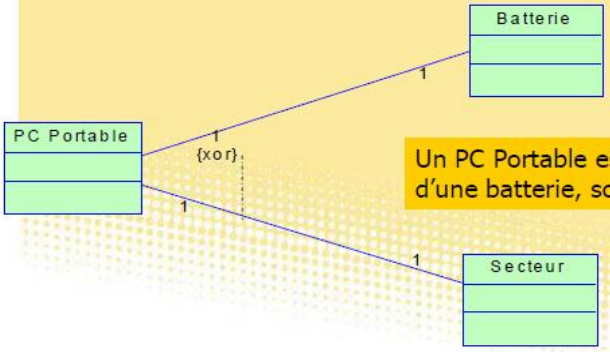
6

Introduction

Contraintes UML sur les associations sans utiliser OCL

Contraintes sur les associations : contrainte {xor}

Xor Indique que parmi un groupe d'associations, une seule est valide à la fois



Un PC Portable est alimenté soit à partir d'une batterie, soit à partir d'un secteur

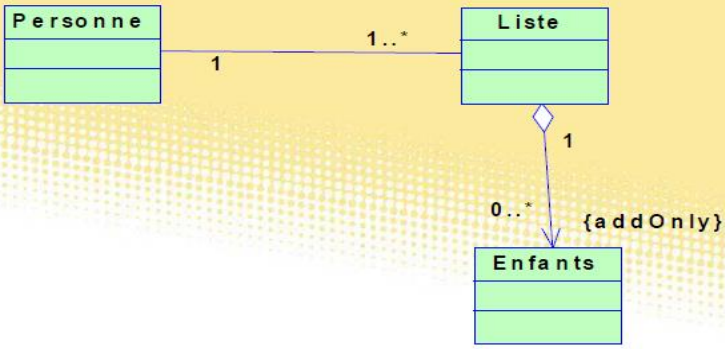
7

Introduction

Contraintes UML sur les associations sans utiliser OCL

Contraintes sur les associations : contrainte {addOnly}

- La contrainte prédéfinie {**addOnly**} autorise l'ajout de nouveaux objets, mais pas leur suppression ni leur mise à jour.



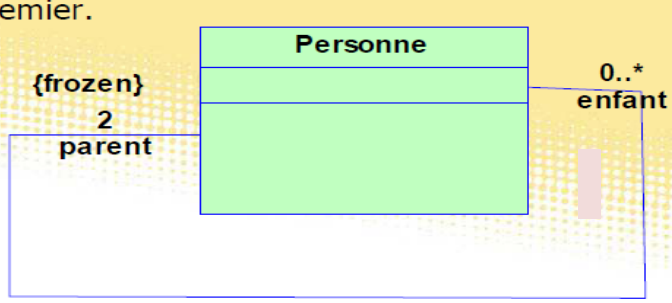
8

Introduction

Contraintes UML sur les associations sans utiliser OCL

Contraintes sur les associations : contrainte {Frozen}

- La contrainte prédéfinie {**frozen**} interdit l'ajout, la suppression ou la mise à jour des liens d'un objet vers les objets de la classe associée, après l'initialisation du premier.



9

Introduction

Contraintes UML sur les associations sans utiliser OCL

UML propose des contraintes, mais ces contraintes reste insuffisante

Solution: OCL (Object Constraint Language)

10

Introduction

Généralité sur OCL

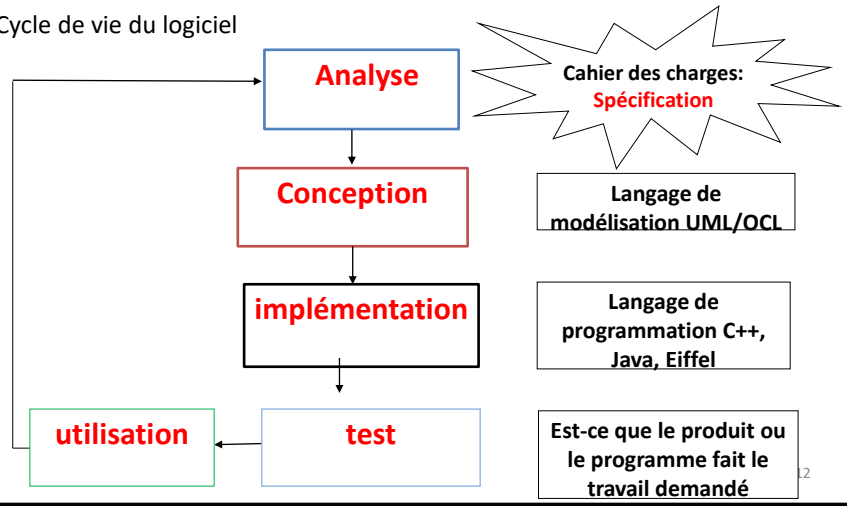
- OCL(Object Constraint Language): en français un langage de contraintes Orienté Objet.
  - définit initialement en 1997 par IBM (Firme international en Informatique côté hardware et Software).
  - a été adapté par OMG (Object Management Group).
  - OMG= joue un rôle liée à la normalisation de la technologie objet.

11

Introduction

Généralité sur OCL

- OCL est un langage de modélisation (ou de spécification) permettant de détecter le plus tôt possible des erreurs dans le cycle de vie du logiciel.
- Cycle de vie du logiciel



12

Introduction

Généralité sur OCL

- Deux utilisations possibles du langage OCL :
  - Pour la conception par contrats : assure la validation de la conception.
  - Pour la manipulation des bases de données c'est-à-dire en tant que le langage de requêtes des BDR (SGBDR).

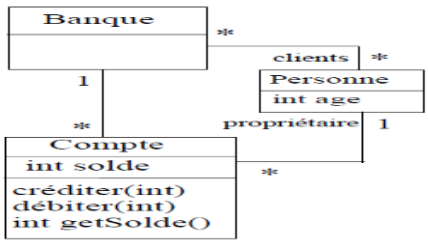
    ➔ **SQL :Structured Query Language**

13

Introduction

Pourquoi OCL?

- **Exemple : Spécification d'une application bancaire**
  - **Un compte doit avoir un solde toujours positif**
  - Un client peut posséder plusieurs comptes
  - Un client peut être client de plusieurs banques
  - **Un client d'une banque possède au moins un compte dans cette banque**
  - Une banque gère plusieurs comptes
  - Une banque possède plusieurs clients



14

Introduction

Pourquoi OCL?

- Manque de précision
  - Le diagramme de classe ne permet pas d'exprimer tout ce qui est défini dans la spécification informelle
- Exemple
  - Le solde d'un compte doit toujours être positif : ajout d'une contrainte sur cet attribut
- Le diagramme de classe permet-il de détailler toutes les contraintes sur les relations entre les classes ?

15

Introduction

Pourquoi OCL?

Diagrammes UML insuffisants

- Pour spécifier complètement une application
  - Diagrammes UML seuls sont généralement insuffisants
  - Nécessité de rajouter des contraintes
- Comment exprimer ces contraintes ?
  - Langue naturelle mais manque de précision, compréhension pouvant être ambigüe
  - Langage formel avec sémantique précise : par exemple OCL
- OCL : Object Constraint Language
  - Langage de contraintes orienté-objet
  - Langage formel (mais simple à utiliser) avec une syntaxe, une grammaire, une sémantique (manipulable par un outil)
  - S'applique entre autres sur les diagrammes UML

16

Introduction

Pourquoi OCL?

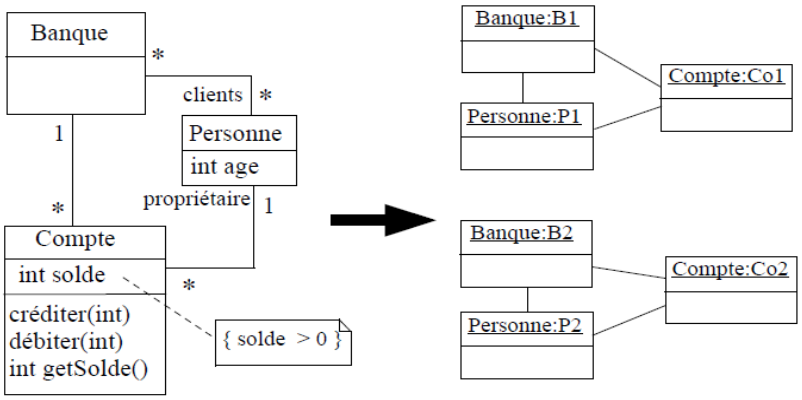
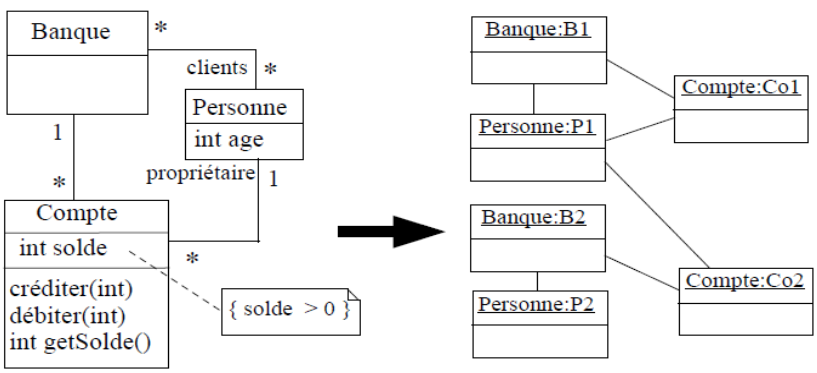


Diagramme d'instances valide vis-à-vis du diagramme de classe et de la spécification attendue

17

Introduction

Pourquoi OCL?

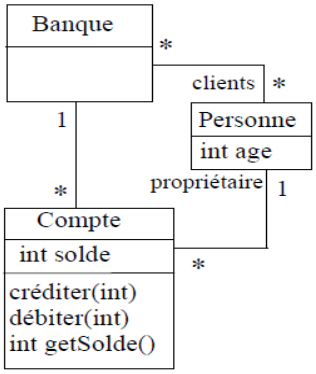


- Diagramme d'instances valide vis-à-vis du diagramme de classe mais ne respecte pas la spécification attendue
  - -Une personne a un compte dans une banque où elle n'est pas cliente
  - -Une personne est cliente d'une banque mais sans y avoir de compte

18

Introduction

Pourquoi OCL?




```
context Compte
inv: solde > 0

context Compte : débiter(somme : Integer)
pre: somme > 0
post: solde = solde@pre - somme

context Compte
inv: banque.clients -> includes (propriétaire)
```

Avantage d'OCL : langage formel permettant de préciser clairement de la sémantique sur les modèles UML

19

Aperçu sur le langage OCL

Utilisation d'OCL dans le cadre d'UML

- OCL peut s'appliquer sur la plupart des diagrammes UML
  - Il sert, entre autres, à spécifier des :
    - Invariants sur des classes
    - Pré et post conditions sur des opérations
    - Gardes sur transitions de diagrammes d'états ou de messages de diagrammes de séquence/collaboration
    - ...

20

Aperçu sur le langage OCL

Contexte

- Une expression OCL est toujours définie dans un contexte
- Ce contexte est l'instance d'une classe
- Mot-clé : **context**
  - Exemple : **context Compte**
- L'expression OCL s'applique à la classe Compte, c'est-à-dire à toutes les instances de cette classe

- ***Invariants***
- Un invariant exprime une contrainte sur un objet ou un groupe d'objets qui doit être respectée en permanence
- Mot-clé : **inv**
  - Exemple : **context Compte**
    inv: solde > 0
- Pour toutes les instances de la classe Compte, l'attribut solde doit toujours être positif

21

Aperçu sur le langage OCL

Pré et post conditions

Pour spécifier une opération

- Précondition : état qui doit être respecté avant l'appel de L'opération (inversement pour la Postcondition : état qui doit être respecté après l'appel de l'opération)
- Mots-clés : **pre et post**

- Dans la postcondition, deux éléments particuliers sont utilisables
  - Attribut **result :** référence la valeur retournée par l'opération
  - mon_attribut**@ pre :** référence la valeur de mon_attribut avant l'appel de l'opération

- Syntaxe pour préciser l'opération:

  Context ma_classe::mon_op(liste_param) : type_retour

22

Aperçu sur le langage OCL

Pré et post conditions

•Exemples : **context Compte::débiter(somme : Integer)**
pre: somme > 0
post: solde = solde@pre – somme

•La somme à débiter doit être positive pour que l'appel de l'opération soit valide

• Après l'exécution de l'opération, l'attribut solde *doit avoir pour* valeur la différence de sa valeur avant l'appel et de la somme passée en paramètre

• **context Compte::getSolde() : Integer**
post: result = solde
•Le résultat retourné *doit être le solde courant*

• **Attention !** On ne décrit pas comment l'opération est réalisée mais des contraintes sur l'état avant et après son exécution

23

Aperçu sur le langage OCL

Accès aux objets, navigation

Dans une contrainte OCL associée à un objet, on peut

- Accéder à l'état interne de cet objet (ses attributs)
- Naviguer dans le diagramme : accéder de manière transitive à tous les objets (et leur état) avec qui il est en relation

•Nommage des éléments

• Attributs ou paramètres d'une opération : utilise leur nom directement

• Objet(s) en association : utilise le nom de la classe associée (en minuscule) ou le nom du rôle d'association du coté de cette classe

• Si cardinalité de 1 pour une association : **référence un objet**

• Si cardinalité > 1 : **référence une collection d'objets**

24

Aperçu sur le langage OCL

Types OCL : types de base

- **Entier**
  opérateurs = <> + - * / abs div mod max min < > <= >=
  - est unaire ou binaire
- **Réel**
  opérateurs = <> + - * / abs floor round max min < > <= >=
  - est unaire ou binaire
- **String**
  opérateurs = **size()**, **concat(String)**,…
  substring(Entier, Entier)
  Les chaînes de caractères constantes s'écrivent entre deux simples quotes : 'voici une chaîne'
- **Booléen**
  opérateurs : **or, xor and not**
  b1 **implies** b2
  if b then expression1 **And**, **or**, **not**, **xor**, **not**, **implies**, **if-then-else**
  - La plupart des expressions OCL sont de types Boolean
  - Notamment les expressions formant les inv, pre et post

25

Aperçu sur le langage OCL

Règles de précédence

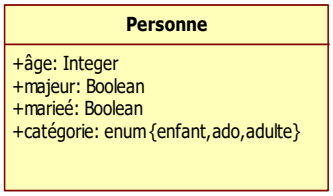
- Ordre de précédence pour les opérateurs/primitives du plus au moins prioritaire
  - @pre
  - . et ->
  - not et -
  - * et /
  - + et -
  - if then else endif
  - >, <, <= et >=
  - = et <>
  - and, or et xor
  - implies
- Parenthèses permettent de changer cet ordre

26

Aperçu sur le langage OCL

Types énumérés

•Leurs valeurs apparaissent précédées de **#**. Par exemple pour donner un invariant sur la classe suivante :




```
context Personne inv :
    if age <=12 then catégorie =#enfant
    else if age <=18 then catégorie =#ado
        else catégorie=#adulte
        endif
    endif
```


Aperçu sur le langage OCL

Nommage de contraintes

Conception par contrat
- précondition→pre→avec étiquette optionnelle
- postcondition→post →avec étiquette optionnelle
- invariant→inv

Exemple:

étiquette

pre somme_positive : $>0

post somme_augmente : solde=solde@pre+s

Commentaire en OCL : utilisation de --

```
context Personne inv:
  if age < 18 -- vérifie age de la personne
  then compte -> isEmpty() -- pas majeur : pas de compte
  else compte -> notEmpty() -- majeur : doit avoir au moins un compte
  endif
```

Aperçu sur le langage OCL

Accès aux objets, navigation

Exemples, dans contexte de la classe Compte

- **solde** : attribut référencé directement
- **banque** : objet de la classe Banque (référence via le nom de la classe) associé au compte
- **propriétaire** : objet de la classe Personne (référence via le nom de rôle d'association) associée au compte
- **banque.clients** : ensemble des clients de la banque associée au compte (référence par transitivité)
- **banque.clients.age** : ensemble des âges de tous les clients de la banque associée au compte
- Le propriétaire d'un compte doit avoir plus de 18 ans

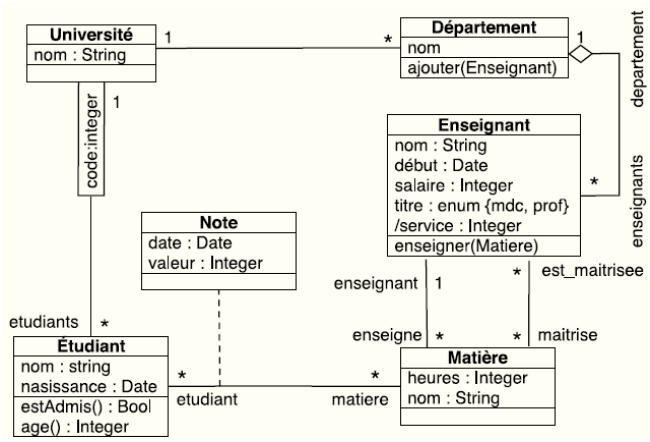
```
context Compte
    inv: propriétaire.age >= 18
```

29

Aperçu sur le langage OCL

Accès aux objets, navigation

Exercice 1: Soit le diagramme de classe suivant. Étendre ce diagramme de classe par des expressions OCL pour assurer les contraintes suivantes :
L'âge d'un étudiant est supérieur à 18 ans
Le salaire minimum d'un enseignant est égal à un million



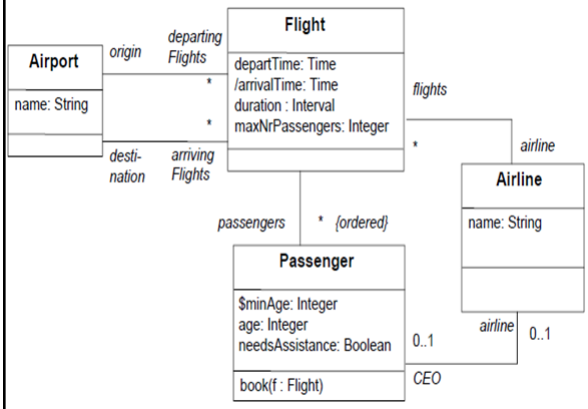
30

Aperçu sur le langage OCL

Accès aux objets, navigation

Exercice 2

Donner la sémantique des expressions OCL suivantes.



context Flight
inv: duration < 4

context Flight
inv: self.maxNrPassengers <= 1000

context Passenger
inv: age >= Passenger.minAge

context Flight
inv: origin <> destination
inv: origin.name = 'Amsterdam'

context Flight
inv: airline.name = 'KLM'

31

Aperçu sur le langage OCL

Opérations sur objets et collections

- OCL propose un ensemble de primitives utilisables sur les collections
  - **size()** : retourne le nombre d'éléments de la collection
  - **isEmpty()** : retourne vrai si la collection est vide
  - **notEmpty()** : retourne vrai si la collection n'est pas vide
  - **includes(obj)** : vrai si la collection inclut l'objet *obj*
  - **excludes(obj)** : vrai si la collection n'inclut pas l'objet *obj*
  - **including(obj)** : la collection référencée doit être cette collection en incluant l'objet *obj*
  - **excluding(obj)** : idem mais en excluant l'objet *obj*
  - **includesAll(ens)** : la collection contient tous les éléments de la collection *ens*
  - **excludesAll(ens)** : la collection ne contient aucun des éléments de la collection *ens*
  - Syntaxe d'utilisation : *objetOuCollection -> primitive*

32

Aperçu sur le langage OCL

Opérations sur objets et collections

- Exemples, invariants dans le contexte de la classe Compte
  - **propriétaire -> notEmpty()** : il y a au moins un objet Personne associé à un compte
  - **propriétaire -> size() = 1** : le nombre d'objets Personne associés à un compte est de 1
  - **banque.clients -> size() >= 1** : une banque a au moins un client
  - **banque.clients -> includes(propriétaire)** : l'ensemble des clients de la banque associée au compte contient le propriétaire du compte
  - **banque.clients.compte -> includes(self)** : le compte appartient à un des clients de sa banque
- **self**: pseudo-attribut référençant l'objet courant

33

Aperçu sur le langage OCL

Relations ensemblistes entre collections

- union
  - Retourne l'union de deux collections
- intersection
  - Retourne l'intersection de deux collections
- Exemples:

  (col1 -> intersection(col2)) -> isEmpty()

  -Renvoie vrai si les collections col1 et col2 n'ont pas d'élément en commun

  col1 = col2 -> union(col3)

  -La collection *col1 doit être l'union des éléments de col2* et de *col3*

34

Aperçu sur le langage OCL

Opérations sur éléments d'une collection

•OCL permet de vérifier des contraintes sur chaque élément d'une collection ou de définir une sous-collection à partir d'une collection en fonction de certaines contraintes

•Primitives offrant ces services et s'appliquant sur une collection ***col***

- **select** : retourne le sous-ensemble de la collection ***col*** *dont les éléments* respectent la contrainte spécifiée
- **reject** : idem mais ne garde que les éléments ne respectant pas la contrainte
- **collect** : retourne une collection (de taille identique) construite à partir des éléments de ***col****. Le type des éléments contenus dans la nouvelle* collection peut être différent de celui des éléments de col.

35

Aperçu sur le langage OCL

Opérations sur éléments d'une collection

- **exists** : retourne vrai si au moins un élément de ***col*** *respecte la* contrainte spécifiée et faux sinon

- **forAll** : retourne vrai si tous les éléments de ***col*** *respectent la contrainte* spécifiée (pouvant impliquer à la fois plusieurs éléments de la collection)

36

Aperçu sur le langage OCL

Opérations sur éléments d'une collection

- Syntaxe de ces opérations : 3 usages
  - **collection -> primitive(expression)**
    - La primitive s'applique aux éléments de la collection et pour chacun d'entre eux, l'expression *expression est* vérifiée. On accède aux attributs/relations d'un élément directement.
  - **collection -> primitive(elt : type | expression)**
    - On fait explicitement apparaître le type des éléments de la collection (ici type). On accède aux attributs/relations de l'élément courant en utilisant elt (c'est la référence sur l'élément courant)
  - **collection -> primitive(elt | expression)**
    - On nomme l'attribut courant (*elt) mais sans préciser son* type

37

Aperçu sur le langage OCL

Opérations sur éléments d'une collection

- Dans le contexte de la classe Banque
  - **compte -> select(c | c.solde > 1000)**
    - Retourne une collection contenant tous les comptes banquaires dont le solde est supérieur à 1000 €
  - **compte -> reject(solde > 1000)**
    - Retourne une collection contenant tous les comptes banquaires dont le solde n'est pas supérieur à 1000 €
  - **compte -> collect(c : Compte | c.solde)**
    - Retourne une collection contenant l'ensemble des soldes de tous les comptes
  - **(compte -> select(solde > 1000)) -> collect(c | c.solde)**
    - Retourne une collection contenant tous les soldes des comptes dont le solde est supérieur à 1000 €

38

Aperçu sur le langage OCL

Opérations sur éléments d'une collection

context Banque

inv: not(clients -> exists (age < 18))

- Il n'existe pas de clients de la banque dont l'âge est inférieur à 18 ans
- not: prend la négation d'une expression

context Personne

inv: Personne.allInstances() -> forAll(p1, p2 |

p1 <> p2 **implies p1.nom <> p2.nom)**

- Il n'existe pas deux instances de la classe Personne pour lesquelles l'attribut nom a la même valeur : deux personnes différentes ont un nom différent
- **AllInstances()**: Primitive s'appliquant sur une classe (et non pas un objet) et retournant toutes les instances de la classe référencée (ici la classe Personne)

39

Aperçu sur le langage OCL

Accès aux attributs pour les collections

- Accès à un attribut sur une collection
- Exemple dans contexte de Banque :

compte.solde

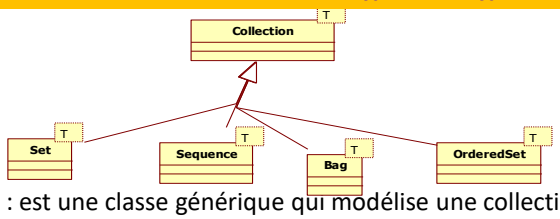
- Renvoie l'ensemble des soldes de tous les comptes
- Forme raccourcie et simplifiée de

compte -> collect (solde)

40

Aperçu sur le langage OCL

Types OCL : types de collection



•**Colection** : est une classe générique qui modélise une collection au sens général. Elle factorise plusieurs opérations telles que : size, count, includes, excludes.

• **Set** : est un classe qui dérive (ou descend) de la classe collection. Elle modélise la notion d'ensemble au sens mathématique. La notion d'ensemble au sens mathématique possède les propriétés suivantes : pas d'ordre et pas de doublons. (ex : Set{1, 3, 5, 4).

• **Sequence** : est une classe qui dérive de la classe colection ayant les deux propriétés suivantes : ordres et avec des doublons:{ 1, 1, 3, 4, 4, 5}

• **Bag** : modélise la notion d'ensemble mathématique mais avec des doublons. (ex : Bag{1,3, 4, 5, 4}).

• **OrderedSet :** modélise la notion d'ensemb,le mathématique avec ordre mais sans doublons: OrderedSet{1, 3,4, 5, 6)

41

Aperçu sur le langage OCL

Appels d'opération des classes

Dans une contrainte OCL : accès aux attributs, objets ... « en lecture »

• Possibilité d'utiliser une opération d'une classe dans une contrainte

- Si pas d'effets de bords (de type « query »)
- Car une contrainte OCL exprime une contrainte sur un état mais ne précise pas qu'une action a été effectuée

•Exemple :

context Banque
inv: compte -> forAll(c | c.getSolde() > 0)

. getSolde() est une opération de la classe Compte. Elle calcule une valeur mais sans modifier l'état d'un compte

42

Aperçu sur le langage OCL

Appels d'opération des classes

Définition de fonctions de type query

• Opérateur « **def** » permet également de définir des fonctions d'interrogation du modèle (query)

- Avec paramètres et type de retour au besoin
- Pour faciliter la navigation et la sélection d'éléments sur le modèle
- Exemple: les comptes dont le solde est supérieur à une certaine somme ***val***

context Banque def: soldesSup(val : Integer) :

Set(Compte) =self.comptes -> select (c | c.solde > val)

• S'utilise ensuite comme une opération de la classe Banque

dont on se sert pour écrire une contrainte

context Banque

inv: self.soldesSup(1000) -> notEmpty()

43

Aperçu sur le langage OCL

Appels d'opération des classes

Propriétés

▪De manière générale en OCL, une propriété est un élément pouvant être:

- Un attribut
- Un bout d'association
- Une opération ou méthode de type requête

• Accède à la propriété d'un objet avec « **.** »

Exemples:

context Compte inv: self.solde > 0

context Compte inv: self.getSolde() > 0

• Accède à la propriété d'une collection avec « **->** »

• On peut utiliser « -> » également dans le cas d'un objet (= collection d'1 objet)

44

Aperçu sur le langage OCL

Appels d'opération des classes

Variables

- Pour faciliter l'utilisation de certains attributs ou calculs de valeurs on peut définir des variables
- Dans une contrainte OCL : let ... in ...
  - **context Personne**

    inv: let argent = compte.solde -> sum() in

    age >= 18 implies argent > 0

  - Une personne majeure doit avoir de l'argent
  - **sum()** : fait la somme de tous les objets de la collection
- Pour l'utiliser partout : def
  - **context Personne**

    def: argent : Integer = compte.solde -> sum()

  - **context Personne**

    inv: age >= 18 implies argent > 0

45

Aperçu sur le langage OCL

Appels d'opération des classes

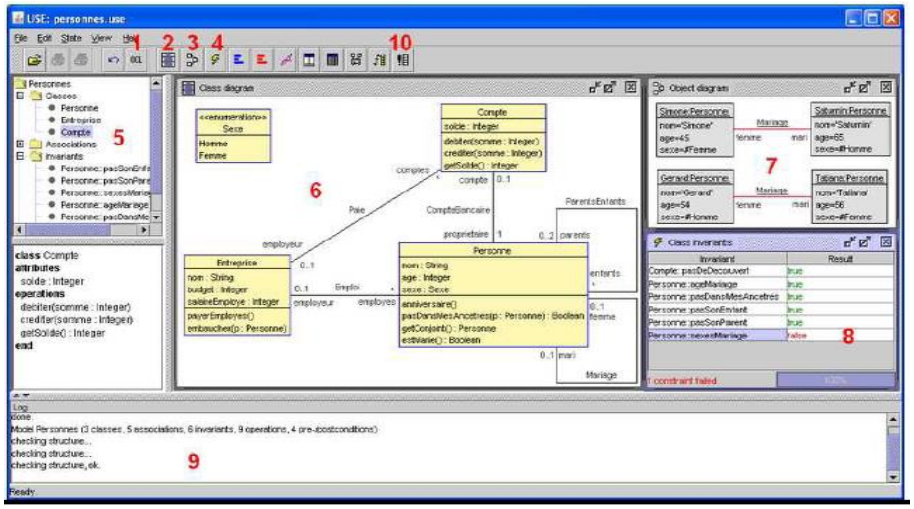
VÉRIFICATION DE CONTRAINTES OCL AVEC USE

- L'outil USE : *a UML based Specification Environnement.*
- USE est un outil permettant de définir des diagrammes de classes UML, des diagrammes d'instance conformes à ces diagrammes de classe et de vérifier des contraintes OCL sur ces instances.
- Outil disponible : http://sourceforge.net/projects/useocl/
- Au lancement de USE, l'interface de l'outil est composée à la fois d'une interface graphique et d'une interface en ligne de commande. Ces deux interfaces permettent de manipuler les mêmes diagrammes chargés par l'outil. L'interface graphique est la suivante :

46

Aperçu sur le langage OCL

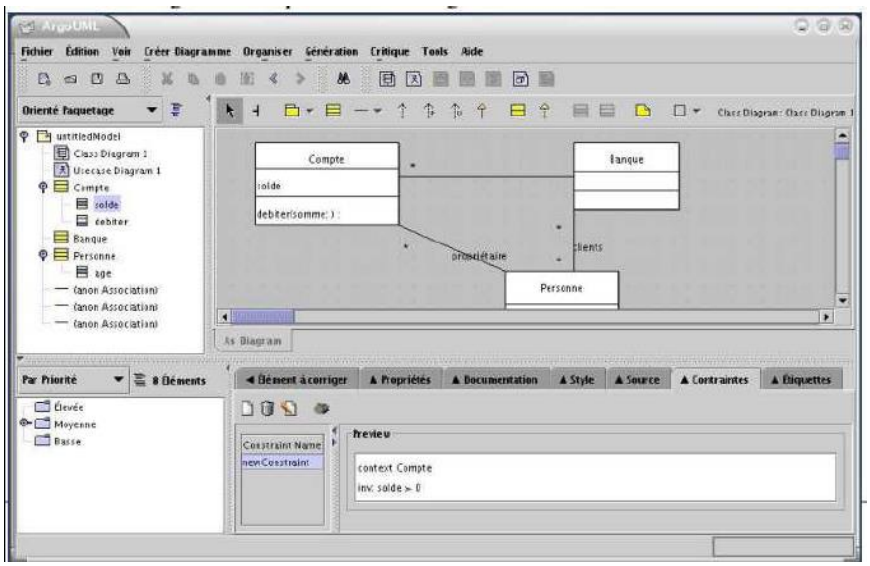
Vérification de contraintes OCL avec USE



47

Aperçu sur le langage OCL

Utilisation en pratique d'OCL lors d'un développement logiciel



Aperçu sur le langage OCL

Utilisation en pratique d'OCL lors d'un développement logiciel

Outils :

- **ArgoUML** : http://argouml.tigris.org/
- **USE** (Mark Richters) UML-based Specification Environment

http ://sourceforge.net/projects/useocl/
Octopus (Warmer & Kleppe)
http://octopus.sourceforge.net/

- **Dresden OCL** http ://dresden-ocl.sourceforge.net/

49

Aperçu sur le langage OCL

Références

-*Cours OCL (Object Constraint Language (OCL)* , Eric Cariou, *Université de Pau et des Pays de l'Adour, UFR Sciences Pau – Département Informatique.*

- *Cours OCL (Object Constraint Language (OCL), Bhiri Mohamed Tahar, Faculté des sciences de Sfax.*

50